A Mini Project Report on

CommuniFind: Hyper-Local Feature Locator

Student of

Information Technology

Pillai College of Arts , Commerce & Science

Submitted

To

Submitted By

Pawani Rishi Devesh

5551

Under Guidance of

Miss. Satvika Guntha

Faculty

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

Mahatma Education Society's

PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE

(Autonomous)

Re-accredited "A" Grade by NAAC (3rdCycle)

Project Completion Certificate

This is to certify that **Pawani Rishi Devesh** of **SYIT A** has completed the project titled "**CommuniFind: Hyper-Local Feature Locator "** under the guidance and supervision during the academic year 2025-26 in the department of **Information Technology**.

Project Guide
Ms. Satvika Guntha

Head of Department
Mrs.Deepika Sharma

Course Coordinator
Mrs.Anju Somani

Pillai College of Arts, Commerce & Science (Autonomous) New Panvel

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Miss. Satvika Guntha, for their valuable guidance, continuous support, and constructive feedback throughout the development of my Mini Project titled "CommuniFind: Hyper-Local Feature Locator." Their mentorship helped me understand the technical complexities of geospatial data processing and Python application development.

I am also grateful to Mrs.Anju Somani, Coordinator of the Department of Information Technology, and the Principal, Dr. Gajanan Wader sir, for providing the necessary academic environment and infrastructure at Pillai College of Arts, Commerce & Science to complete this project successfully.

I would like to thank the teaching and non-teaching staff of the Department of Information Technology for their cooperation.

Finally, I wish to thank my parents and friends for their encouragement and support, which helped me stay motivated throughout this endeavor.

Rishi Pawani

Class: SYIT - A

Roll No.: 5551

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

TABLE OF CONTENTS

Project Completion Certificate **2**
ACKNOWLEDGEMENT **3**
TABLE OF CONTENTS **4**
LIST OF FIGURES 6
LIST OF TABLES 7
SYMBOLS & ABBREVIATIONS 7
ABSTRACT **8**
1. INTRODUCTION **9**
1.1 PROBLEM STATEMENT **9**
1.2 OBJECTIVES **10**
1.3 SCOPE OF PROJECT **11**
2. LITERATURE REVIEW **12**
1. Google Maps 12
2. OpenStreetMap (OSM) 12
3. ArcGIS (ESRI GIS Solutions) 13
4. GeoDa 13
5. Neighborhood & Community Apps (Nextdoor, Local Social Platforms) 13
3. STUDY AREA **14**
1. LOCATION AND EXTENT 14
2. GENERAL DESCRIPTION OF STUDY AREA 14
4. MATERIALS AND METHODS **15**
1. TOOLS AND TECHNOLOGIES 15
2. RESEARCH METHODOLOGY **16**
1. Problem Identification and Baseline Study 16
2. Requirement Analysis and Specification 16
3. Data Collection and Manual Curation 16
4. System Design and UML Modeling 17
5. Technology Selection and Justification 17
6. System Implementation (Modular Approach) 17
7. Testing and Validation 18
8. Evaluation and Comparative Analysis 18
9. Documentation and Reporting 18
3. DIAGRAMS **19**
E.R Diagram: **19**
1. Steps in Analysis/Activity Diagram: **20**

2. Use-Case Diagram: **21**
3. Class Diagram: **22**
4. Sequence Diagram: **23**
5. Deployment Diagram: **24**
6. Package Diagram: **25**
7. Component Diagram: **26**
8. State Machine Diagram: **27**
9. Object Diagram: **28**
10. Composite Structure Diagram: **29**
5. IMPLEMENTATION **30**
6. RESULTS **33**
7. CONCLUSION **39**
FUTURE DEVELOPMENTS 40
REFERENCES / BIBLIOGRAPHY **41**

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

LIST OF FIGURES

- **Figure 1:** E.R Diagram ……18
- **Figure 2:** Steps in Analysis (Activity Diagram) …… 19
- **Figure 3:** Use-Case Diagram ……20
- **Figure 4:** Class Diagram ……21
- **Figure 5:** Sequence Diagram ……22
- **Figure 6:** Deployment Diagram ……23
- **Figure 7:** Package Diagram ……24
- **Figure 8:** Component Diagram ……25
- **Figure 9:** State Machine Diagram ……26
- **Figure 10:** Object Diagram ……27
- **Figure 11:** Composite Structure Diagram ……28
- **Figure 12:** Application Landing Page providing user authentication options.…….33
- **Figure 13:** Built-in User Guide explaining application functionality.…….33
- **Figure 14:** User Registration Module with Region Selection.…….34
- **Figure 15:** Onboarding Tutorial for first-time users……34
- **Figure 16:** Home Dashboard showing Search Interface and Dynamic Safety Tips.…….35
- **Figure 17:** Search Results illustrating the Relative Plotting Algorithm with distance and bearing calculation.…….35
- **Figure 18:** Error Handling and User Feedback mechanism.…….36
- **Figure 19:** Crowdsourcing Interface allowing users to contribute data.…….36
- **Figure 20:** Administrator Panel for moderating user-submitted location requests.…….37
- **Figure 21:** Administrative controls for managing registered users and location databases.…….37
- **Figure 22:** *The CommuniFind backend database schema visualized in MongoDB Compass*.……38

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

LIST OF TABLES

- **Table 1:** Tools and Technologies ……14

SYMBOLS & ABBREVIATIONS

Abbreviation

1. **GIS :** Geographic Information System
2. **GUI :** Graphical User Interface
3. **GPS :** Global Positioning System
4. **API :** APplication Programming Interface
5. **CSV :** Comma Separated Values
6. **UML :** Unified Modeling Language
7. **BMC :** Brihanmumbai Municipal Corporation
8. **OSM :** Open Street Map

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

ABSTRACT

CommuniFind is a geospatial data processing utility designed to solve the acute challenge of accessing highly specific community resources within dense, complex urban environments like **Mumbai**. While major mapping applications handle commercial points of interest, they fail dramatically when a resident or visitor needs hyper-local, non-commercial amenity data, such as "a neighborhood park with a dedicated **cricket pitch**," "a public library section for **Marathi literature**," or the location of a reliable, non-franchised **Vada Pav stall** or essential **Public Toilet for Women**.

Built using **Python** and **Tkinter**, CommuniFind operates entirely as a standalone, offline application, circumventing issues with unreliable connectivity and respecting user privacy. Its core function is precise tag-based filtering against a pre-compiled local database of Mumbai amenities. It uses the geopy library to calculate the exact distance (in Kilometers) and the directional bearing to matching locations relative to the user's input coordinates. The results are visualized using Matplotlib on a simple, custom-drawn scatter plot, avoiding complex map APIs. This unique approach provides immediate, actionable, and privacy-conscious direction to essential local infrastructure, empowering citizens to navigate and utilize their local resources effectively.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

1. INTRODUCTION

1.1 PROBLEM STATEMENT

The Challenge of Granularity in Indian Megacities
Mumbai's rapid development and high population density create unique challenges for digital resource location. The generalization inherent in standard mapping tools creates a significant "Local Information Gap" for specific public needs:

1. Neglect of Non-Commercial/Public Amenities

Critical public services and community features—such as specific BMC Ward Offices, accessible public ramps, low-cost *Tiffin* distribution points, or detailed features within public gardens (e.g., specific entry gates or designated *Senior Citizen corners*)—are either missing or have non-standardized digital representations.

2. Ambiguity in Address and Location Data

Many locations in older or unplanned areas of Mumbai lack formal street addresses, relying on landmarks or local names. This makes automated search unreliable. A location based on a detailed tag (e.g., "near the old clock tower") is often more useful than a standard pin drop.

3. Connectivity Dependence and Cost

Reliance on cloud-based maps is problematic in areas with spotty mobile data connectivity or for users conscious of data usage costs. An offline solution is essential for reliable access.

4. Need for Language and Culture-Specific Tags

The crucial details often relate to local culture or language (e.g., a specific *Dabbawala* drop-off point, or a *Ganesh Murti Visarjan* route detail). These highly specific tags are impossible to find using global search indexes.
CommuniFind directly tackles the complexity and density of the Mumbai urban environment by leveraging manually verified, tag-rich, and locally stored data, providing a specialized discovery tool that standard global platforms cannot replicate.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

1.2 OBJECTIVES

The primary objective of CommuniFind is to empower residents with precise, accessible, and private information regarding their local environment. The specific, measurable objectives are as follows:

1. User Interface Accessibility and Input

Design and implement a Graphical User Interface (GUI) using Tkinter that is simple, intuitive, and highly legible. The interface must be optimized for receiving coordinates and descriptive keyword **tags** (via standard English transliteration).

2. Local Data Integration and Filtering

Establish a mechanism to query a local MongoDB collection containing structured Mumbai location data (Name, Latitude, Longitude, Localized Tags). The system must efficiently filter this data based on multiple, concurrent, comma-separated keyword tags provided by the user.

3. Accurate Geospatial Calculation (Metric)

Implement the necessary functions (via the geopy library) to accurately calculate the straight-line distance in Kilometers (KM) and the directional bearing (e.g., North, Southeast) from the user-provided location to every matching resource.

4. Custom Relative Data Visualization

Develop a custom visualization module using Matplotlib to generate a simple 2D scatter plot. This plot must clearly display the user's location at the origin (center) and plot all matching resource locations relative to the user's position, providing an intuitive directional reference without complex map rendering.

5. Offline Operation and Data Privacy

Ensure the entire application—including data storage, calculation, and visualization—operates independently of the internet. The system must use a local dataset, guaranteeing complete privacy and eliminating reliance on external APIs or location tracking.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

1.3 SCOPE OF PROJECT

The scope of CommuniFind is tightly defined to achieve the core objective—hyper-local feature location and visualization—specifically within the context of Mumbai's unique data landscape.

Functional Scope (Inclusions)

1. **Coordinate Input:** Allow the user to select a starting hub from a predefined list of major Mumbai landmarks.
2. **Tag Filtering:** Implement a search function that accepts one or more tags (e.g., park, cricket, shade) and filters the internal data to only include resources matching *all* specified tags.
3. **Metric Calculation:** Calculate and display the precise straight-line distance in **Kilometers (KM)** and the cardinal direction (e.g., North-East) relative to the user.
4. **Visualization:** Generate a scatter plot showing the user at the center (0,0) and placing the matching resources based on their relative distance and direction. Labels must be clear and legible.
5. **Result Display:** Present a list view of matching resources alongside the visualization, detailing the Name, Distance (in KM), and all associated Tags.

Project Limitations (Exclusions)

1. **No GPS/Geolocalization:** User location must be manually input or selected from a pre-defined list; automatic detection is excluded.
2. **No Live Mapping/Routing:** The tool will *not* provide turn-by-turn directions or integrate with live external map services.
3. **Data Curation:** The project relies on a static, pre-populated local MongoDB. The process of gathering, cleaning, and verifying the specific Mumbai amenity data is outside this development scope.
4. **No Complex Address Handling:** The system relies on the accuracy of the Latitude/Longitude data in the database; it does not attempt to parse complex, non-standard Indian street addresses.

Future Scope / Enhancements (Mumbai Specific)

1. Integration of an optional feature to translate common search terms (e.g., 'Toilet' to 'Sauchalaya') before filtering the tags.
2. Allow users to quickly save and recall their last five known locations (e.g., Home, Office).

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

2. LITERATURE REVIEW

With the rapid growth of urban populations, digital location-based services have become essential for navigating cities and accessing nearby resources. Several commercial and open-source geospatial systems exist today; however, most of them focus on generalized or commercial points of interest and fail to address the need for **hyper-local, non-commercial, and culturally contextual community resources**, especially in dense Indian cities like Mumbai. This section reviews existing systems and compares them with the proposed *CommuniFind* application.

1. Google Maps

Google Maps is the most widely used navigation and location discovery platform, offering real-time routing, business listings, and satellite imagery.

Limitations:

Despite its scale, Google Maps primarily emphasizes commercial establishments and standardized addresses. Many hyper-local public amenities such as small parks, informal food stalls, women-specific public toilets, or culturally significant landmarks are either missing, inaccurately labeled, or unsearchable through descriptive tags. Additionally, Google Maps is entirely cloud-based, requiring continuous internet connectivity and collecting user location data.

How CommuniFind Improves:

CommuniFind operates **offline**, uses **manually curated local datasets**, and supports **tag-based discovery** (e.g., “cricket”, “shade”, “women-only”). It provides privacy-preserving, granular access to community infrastructure that Google Maps does not prioritize.

2. OpenStreetMap (OSM)

OpenStreetMap is a collaborative, open-source mapping platform that provides raw geographic data used by many applications.

Limitations:

While OSM contains detailed spatial data, it primarily serves as a **data repository** rather than a user-friendly discovery tool. Searching OSM requires technical knowledge, external software, or web interfaces, making it unsuitable for everyday users seeking quick, filtered, distance-based results.

How CommuniFind Improves:

CommuniFind transforms static location data into a **purpose-built discovery application** by integrating distance calculations, bearing directions, and visual output through an intuitive GUI—without requiring technical GIS knowledge.

3. ArcGIS (ESRI GIS Solutions)

ArcGIS is an industry-grade Geographic Information System used by governments and enterprises for spatial analysis and mapping.

Limitations:

ArcGIS tools are expensive, complex, and designed for professionals. They are excessive for simple community-level queries and unsuitable for offline, lightweight usage by citizens or students. Licensing and training costs further restrict accessibility.

How CommuniFind Improves:

CommuniFind provides a **cost-free, lightweight alternative** for basic geospatial analysis using Python libraries, making spatial insights accessible to non-experts without sacrificing accuracy for distance and direction calculations.

4. GeoDa

GeoDa is an open-source desktop application focused on spatial data analysis and statistical exploration.

Limitations:

GeoDa primarily targets researchers and analysts, emphasizing spatial statistics rather than user-centric resource discovery. Its interface and workflow are not optimized for searching amenities using descriptive or culturally relevant tags.

How CommuniFind Improves:

CommuniFind focuses on **practical usability**, enabling users to input coordinates and natural tags to retrieve actionable results. The visualization is intentionally simple, prioritizing clarity over statistical complexity.

5. Neighborhood & Community Apps (Nextdoor, Local Social Platforms)

Applications like Nextdoor aim to connect people within neighborhoods for communication and local updates.

Limitations:

These platforms are social rather than analytical. They do not provide structured geospatial filtering, distance computation, or visual mapping of amenities. Data is often unstructured, subjective, and not reusable for systematic analysis.

How CommuniFind Improves:

CommuniFind offers a **structured, data-driven approach** to community discovery, ensuring repeatable searches, measurable distances, and visual clarity, making it suitable for civic planning, research, and informed decision-making.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

3. STUDY AREA

1. LOCATION AND EXTENT

The study area for this project is the **Greater Mumbai (Bombay)** metropolitan region, Maharashtra, India. Geographically, Mumbai is situated on the Konkan coast on the west coast of India and has a deep natural harbour. The city spans a total area of approximately **603.4 square kilometers** (233 sq mi).

The extent of the "CommuniFind" application's dataset initially focuses specifically on the high-density urban wards of Mumbai, where the complexity of infrastructure is highest. The geospatial logic covers the coordinates ranging roughly from **Latitude 18.89° N to 19.27° N** and **Longitude 72.77° E to 72.98° E**. This region is characterized by an extremely high population density (approx. 21,000 people per square kilometer), which creates the specific navigational challenges this project aims to solve.

2. GENERAL DESCRIPTION OF STUDY AREA

Mumbai's rapid development and unique urban planning create a distinct set of challenges for digital resource location, which standard mapping tools often fail to address.

1. The "Granularity Gap" The study area is defined by a mix of formal commercial zones and informal, high-density residential clusters. While major mapping applications (like Google Maps) effectively catalog commercial points of interest, they often neglect non-commercial and public amenities that are vital for local residents. These include specific **BMC Ward Offices**, **accessible public ramps**, **Pyaavs (traditional water fountains)**, and **designated Senior Citizen corners** in public gardens.

2. Address Ambiguity A significant characteristic of the study area is the lack of formal street addresses in many older or unplanned localities. Navigation in Mumbai often relies on landmarks, local names, or descriptive tags (e.g., "near the old clock tower") rather than standardized house numbers. This necessitates a tag-based search system rather than a traditional address-based search.

3. Digital Connectivity Issues Despite being a major metropolitan area, several pockets within the study area experience inconsistent mobile data connectivity. Furthermore, for a significant portion of the population, data costs are a constraint. This characteristic of the study area dictates the need for an **offline-first** solution that does not rely on continuous cloud connectivity to function.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

4. MATERIALS AND METHODS

1. TOOLS AND TECHNOLOGIES

The technology stack for CommuniFind is chosen for its suitability in creating a robust, stable, and beginner-friendly standalone desktop application for the localized data challenge.

Component	Tool / Library	Role in Project	Mumbai Context Rationale
Programming Language	**Python 3**	Core language for logic, data processing, and scripting.	High versatility and ease of use for quick prototyping of data utilities.
User Interface	**CustomTkinter(Wrapper for tkinter)**	Python's standard, built-in GUI toolkit.	Creates a lightweight, stable UI essential for an offline application, avoiding web complexities.
Geospatial Calculation	**geopy**	Used for accurate distance calculation in Kilometers and directional bearing determination (Haversine formula).	Handles the spherical math required to calculate real-world distances between two latitude/longitude points efficiently.
Data Handling	**Pandas Module**	Manages the reading, filtering, and manipulation of the structured local data .	Crucial for quickly searching and filtering large, tabular data sets based on multiple keyword tags.
Visualization	**Matplotlib**	Generates the custom 2D scatter plot visualizing resource locations.	Allows the creation of a simple, non-API-dependent map that only displays relevant resource pins relative to the user.
Data Storage	**MongoDB**	Stores all resource locations, coordinates, and descriptive tags (e.g., 'cricket', 'public-toilet', 'BMC').	Guarantees offline capability, simplicity, and complete data privacy for users in challenging areas

2. RESEARCH METHODOLOGY

The research methodology for **CommuniFind** follows a systematic engineering approach, combining geospatial analysis with user-centered software design. The process is divided into the following nine phases:

1. Problem Identification and Baseline Study

The research began with a comparative analysis of existing Global Information Systems (GIS) like Google Maps and OpenStreetMap.

- **Gap Analysis:** While these platforms are excellent for commercial routing, the study identified a significant "granularity gap" in Mumbai.
- **Mumbai Specificity:** Observations showed that essential local markers—such as specific *BEST* bus stop niches, *Pyaavs* (traditional water fountains), or specific BMC ward counters—are often missing or lack the "searchable tags" needed by residents.

2. Requirement Analysis and Specification

A formal requirement gathering phase was conducted to ensure the tool met the unique needs of a Mumbai user.

- **Functional Requirements:** The system must support multi-keyword filtering (e.g., "Marathi, Library, Quiet"), manual coordinate entry (to save battery/data), and relative direction plotting.
- **Non-Functional Requirements:** High priority was placed on **Zero-Connectivity Dependency** (total offline use) and **Data Sovereignty** (no location data leaves the local machine).

3. Data Collection and Manual Curation

Since standard APIs lack the required detail, a custom data strategy was implemented.

- **Curation Strategy:** Data was manually curated by simulating local needs. This involved identifying "Hyper-Local Amenities" (e.g., *Sulabh Sauchalayas*, localized community centers, and specialized markets).
- **Data Structure:** The data was normalized into a MongoDB database. Each entry consists of: Name, Latitude, Longitude, Category, and Tags. The tags use local terminology (e.g., "Maidan," "Mandir," "Station-East") to reflect how Mumbaikars actually describe their surroundings.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

4. System Design and UML Modeling

To ensure structural integrity, the system was modeled using the Unified Modeling Language (UML):

- **Behavioral Models:** *Use Case Diagrams* defined user interactions (Setting Location, Searching Tags). *Activity Diagrams* mapped the flow from input to distance calculation.
- **Structural Models:** *Class Diagrams* define the relationship between the DataManager, GeoCalculator, and UIRenderer. *Sequence Diagrams* trace the logic flow when a user hits "Search," ensuring the Matplotlib plot updates only after the Geopy calculation is complete.

5. Technology Selection and Justification

The "Python-centric" stack was selected based on the following research criteria:

- **Python:** Chosen for its rapid development cycle and robust handling of String data.
- **Geopy:** Selected for its precision in implementing the **Haversine Formula**, essential for calculating "as-the-crow-flies" distances in dense urban grids.
- **Matplotlib:** Used to solve the visualization problem without needing heavy tile-based map engines.
- **Pandas:** Utilized for high-speed "Vectorized" filtering, allowing the app to search through thousands of local points in milliseconds.

6. System Implementation (Modular Approach)

Implementation was carried out using a modular architecture:

- **The Input Module:** Handles coordinate validation (ensuring Lat/Long are within Indian bounds).
- **The Search Engine:** A Python-based filtering logic that performs intersection checks on tags.
- **The Geo-Logic:** Converts raw coordinates into distance (KM) and Bearing (Degrees), which is then converted into cardinal directions (N, S, E, W).
- **The Radar View:** A custom Matplotlib scatter plot that uses polar or Cartesian projections to show the user at the center.

7. Testing and Validation

A rigorous testing protocol was established:

- **Unit Testing:** Individual functions for distance calculation were tested against known distances between Mumbai landmarks (e.g., Gateway of India to Chhatrapati Shivaji Maharaj Terminus).
- **Functional Testing:** Testing "AND" logic in tags (e.g., searching for "Park" + "Gym" must only return locations with both).
- **Edge Case Handling:** Validating system behavior when no matches are found or when the user enters non-numeric coordinates.

8. Evaluation and Comparative Analysis

The system was evaluated against the initial problem statement:

- **Accuracy:** How closely does the Matplotlib plot reflect real-world orientation?
- **Granularity:** Can the user find a specific "Aarey Milk Centre" that Google Maps misses?
- **Performance:** Measurement of the "Search-to-Plot" time, ensuring it remains under 1 second for a seamless experience.

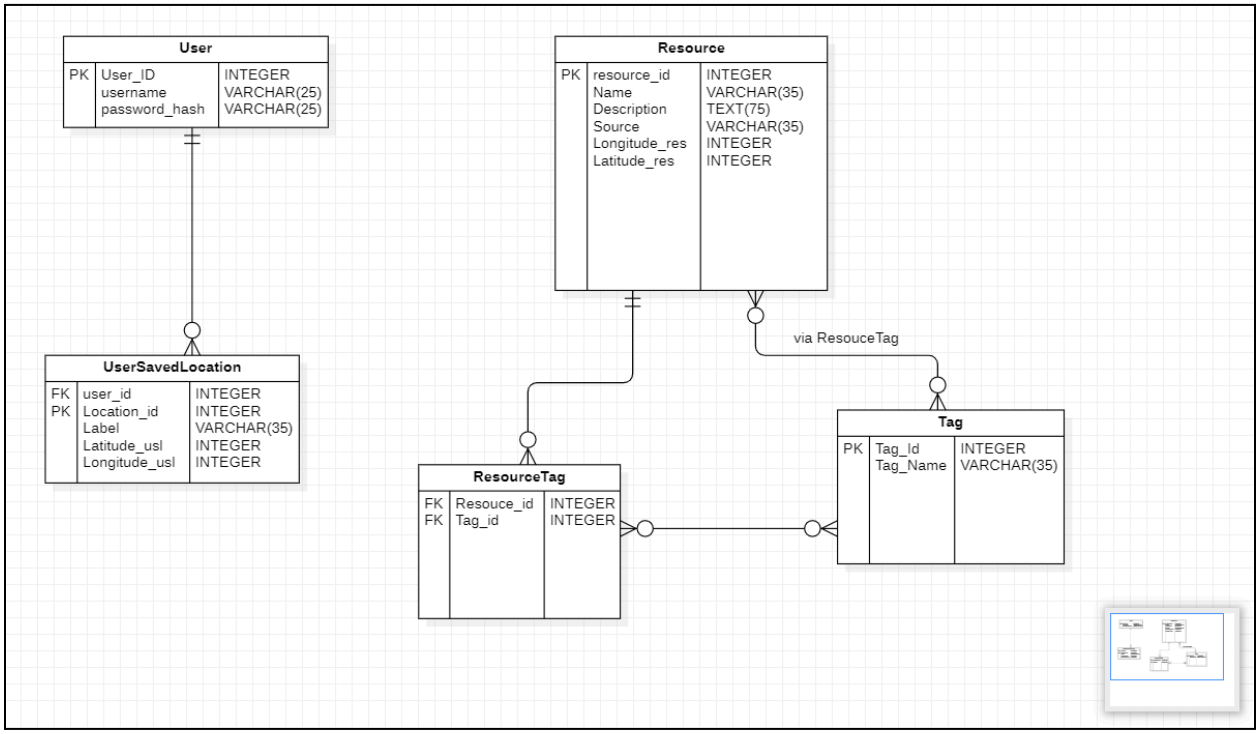
9. Documentation and Reporting

The final phase involved synthesizing the research into a technical report. This includes the documentation of the code architecture, the final UML diagrams, and an analysis of how CommuniFind serves as a privacy-preserving alternative to cloud-based maps for the city of Mumbai.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

3. DIAGRAMS

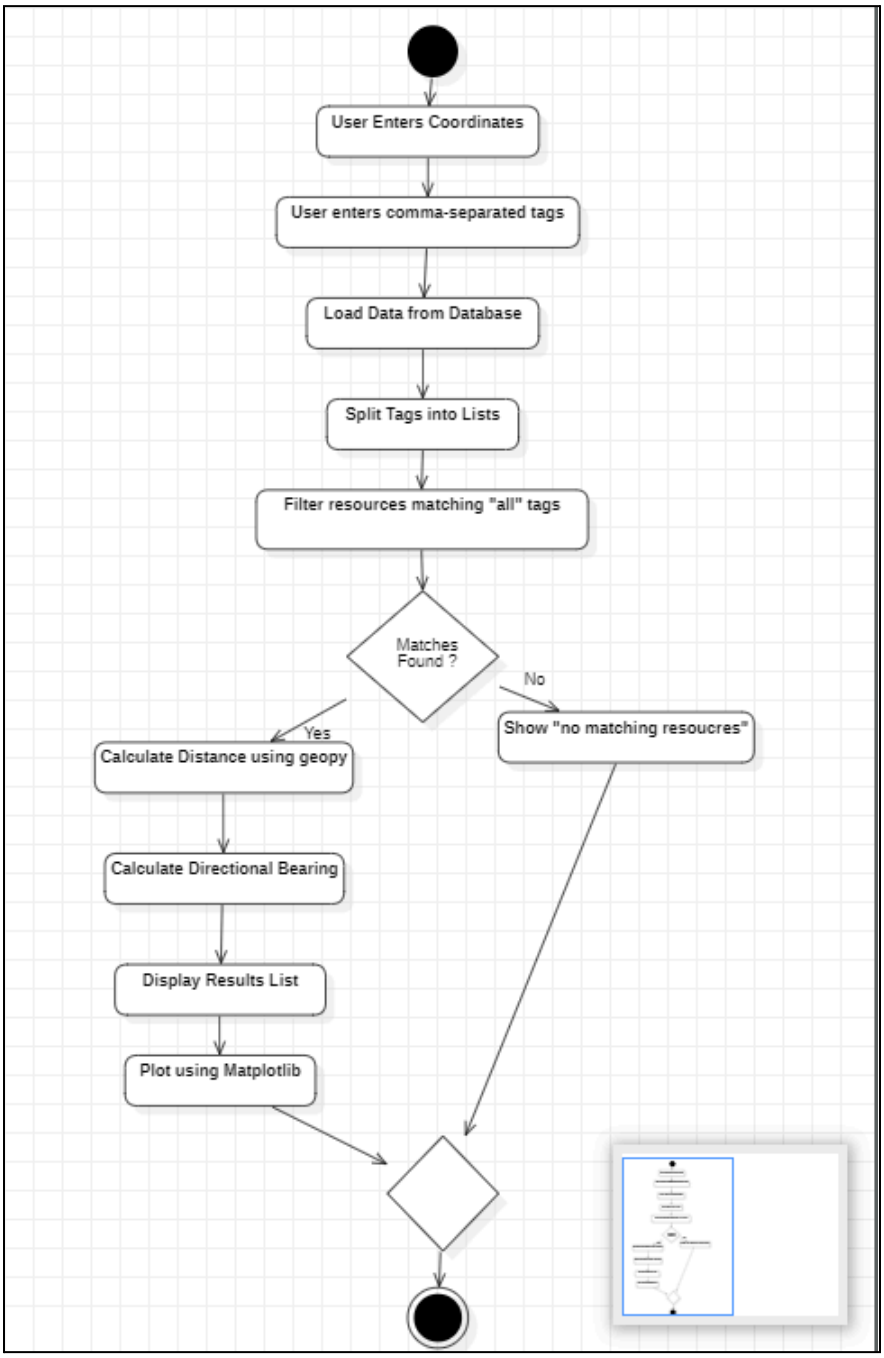
E.R Diagram:



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

1. Steps in Analysis/Activity Diagram:

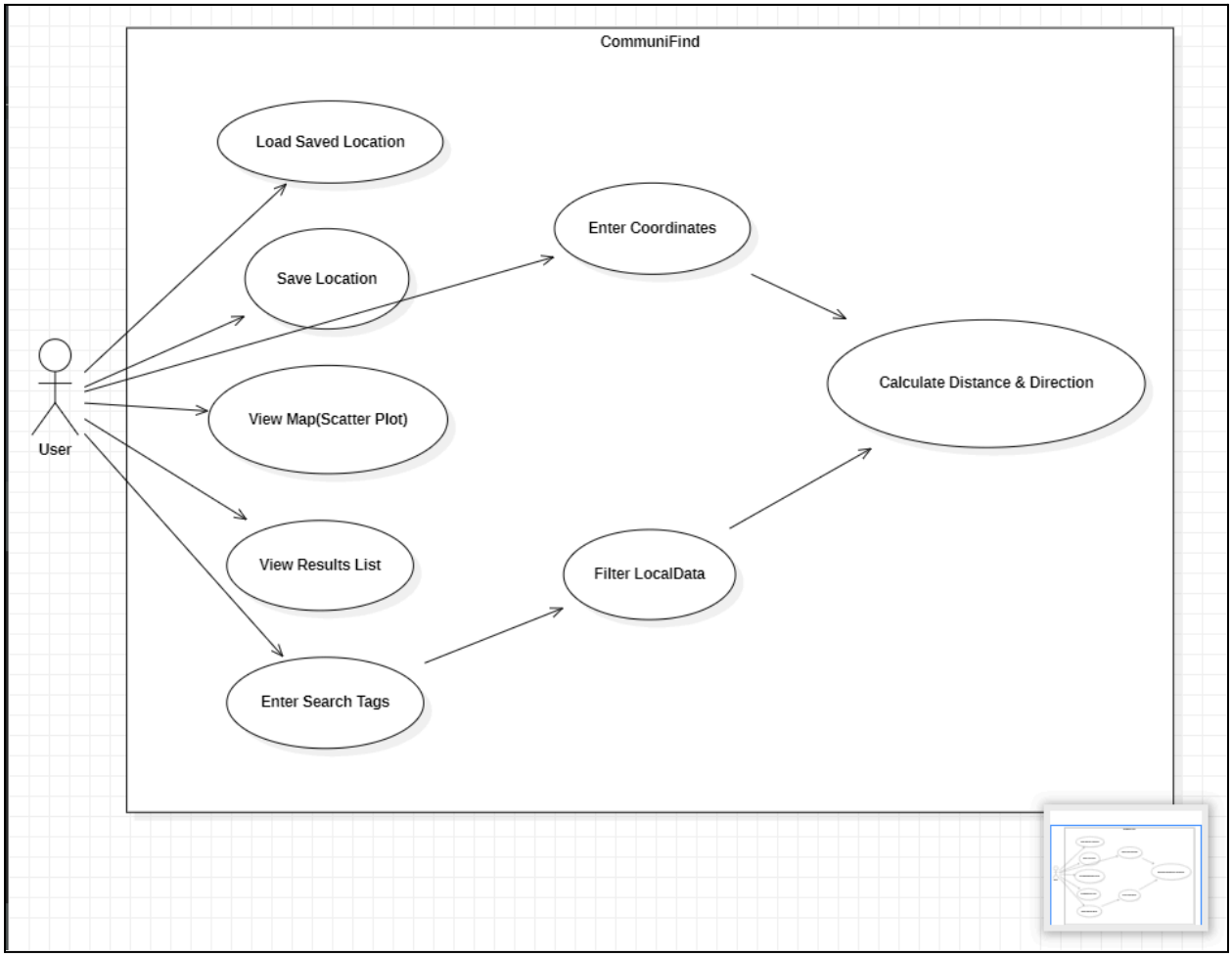
The diagram shows a workflow where a user enters coordinates and tags, and the system loads data, splits tags, and filters resources that match all tags.
If matches exist, it calculates distance and bearing, displays results, and plots them.
If no matches are found, it simply shows a “no matching resources” message.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

2. Use-Case Diagram:

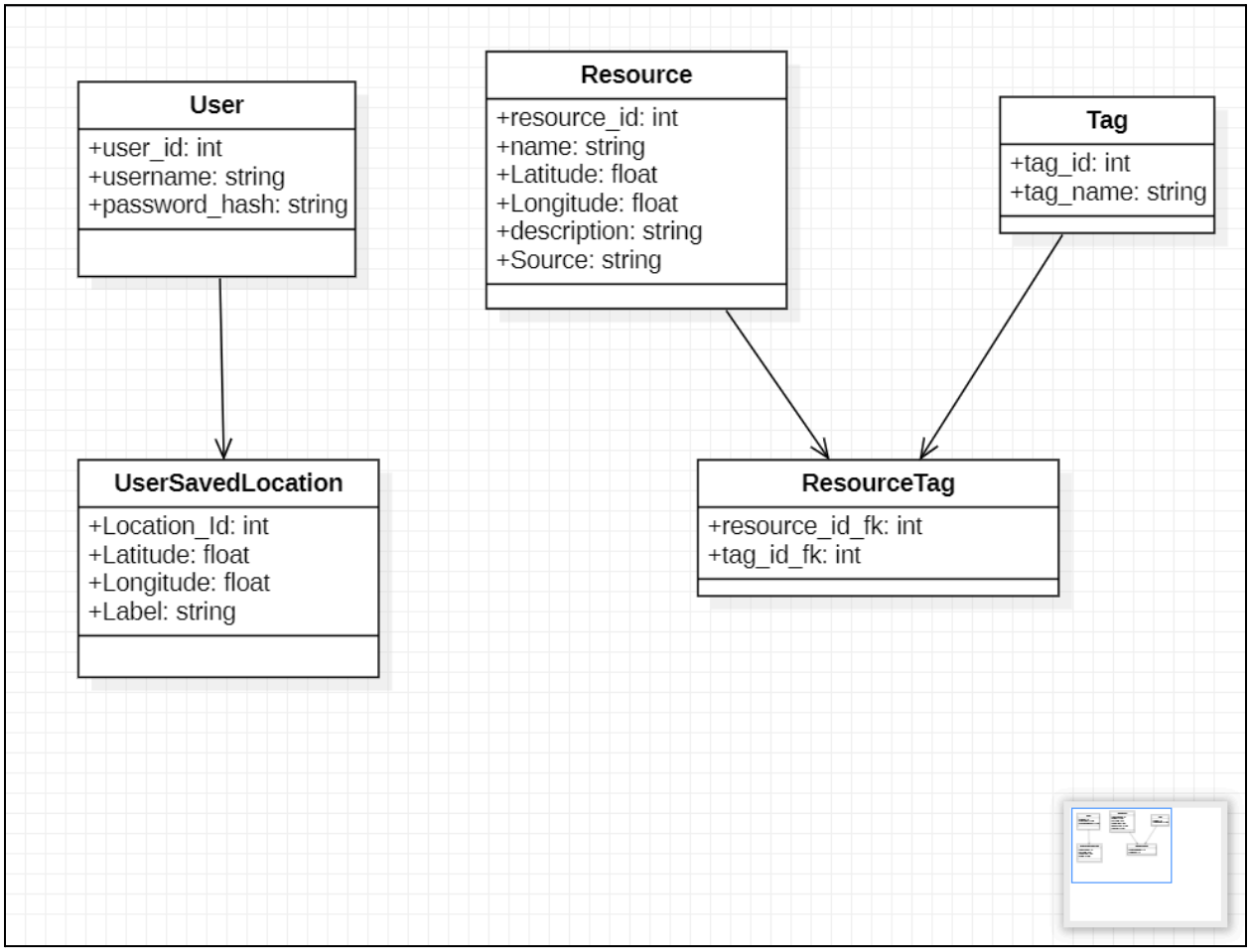
The use-case diagram shows a user interacting with the CommuniFind system to enter coordinates and search tags, filter local data, and compute distance and direction.
The user can also save or load locations, view a results list, and view a map (scatter plot).
Each action represents a system function that supports finding and visualizing nearby resources.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

3. Class Diagram:

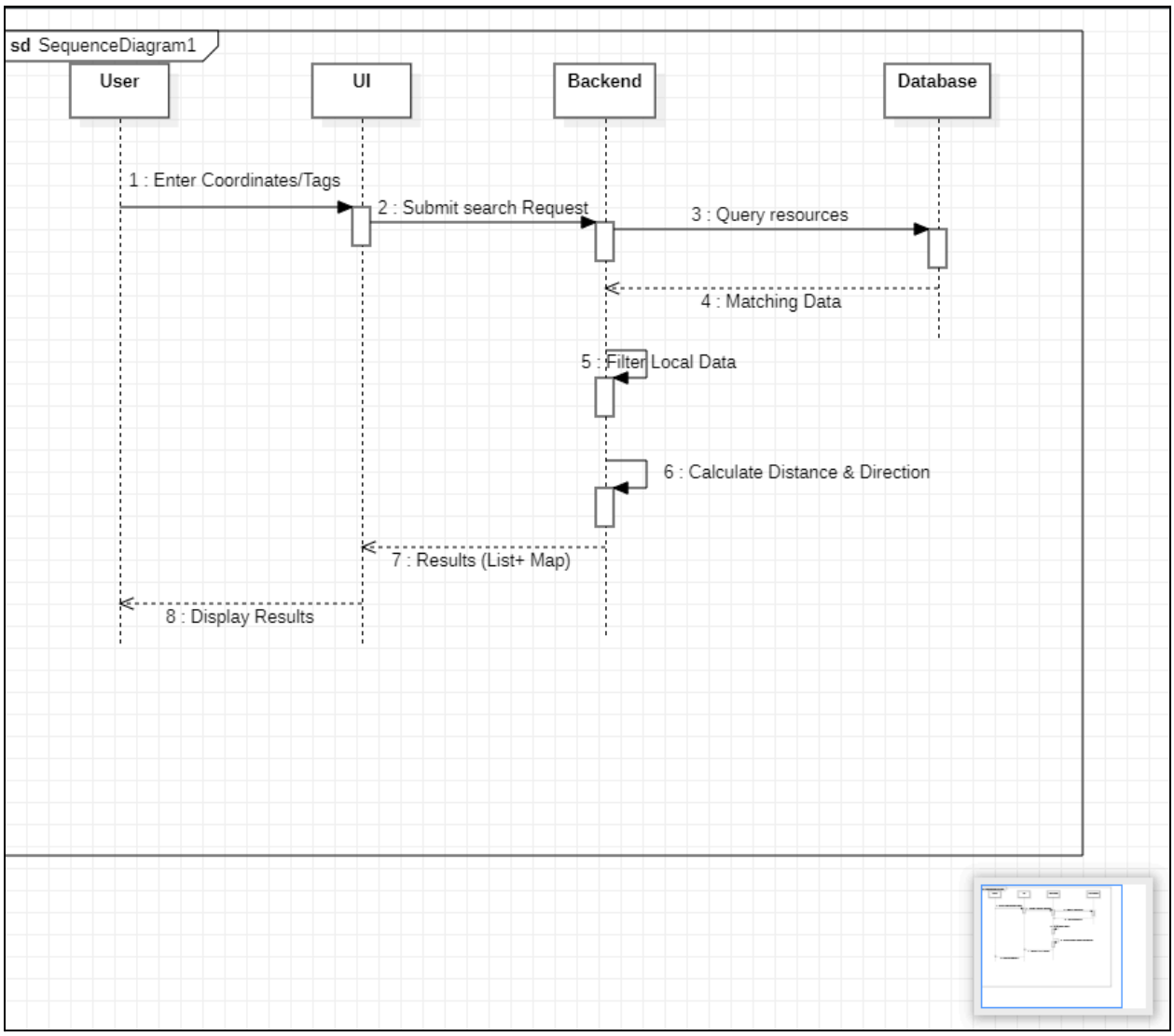
The diagram shows a system where Users can save specific Locations, and Resources can be tagged with multiple Tags through a linking table ResourceTag.
Each entity stores its own attributes (e.g., coordinates for locations/resources).
Relationships connect users to their saved locations and resources to their associated tags.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

4. Sequence Diagram:

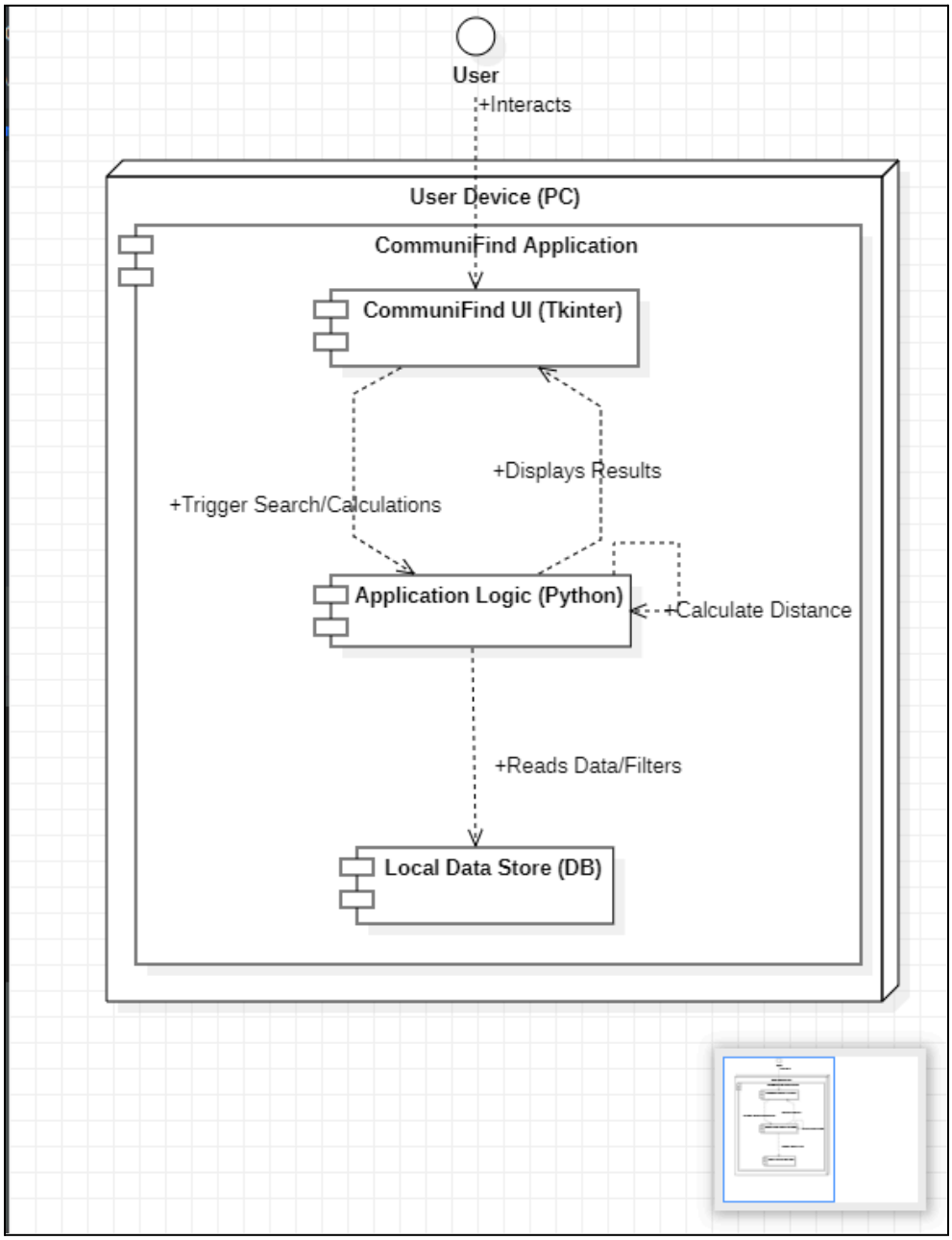
The User interacts with the UI to submit a search request (Coordinates/Tags).
The Backend queries the Database for matching resources, then filters and calculates distance/direction.
The results (List + Map) are sent back to the UI for the User to see.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

5. Deployment Diagram:

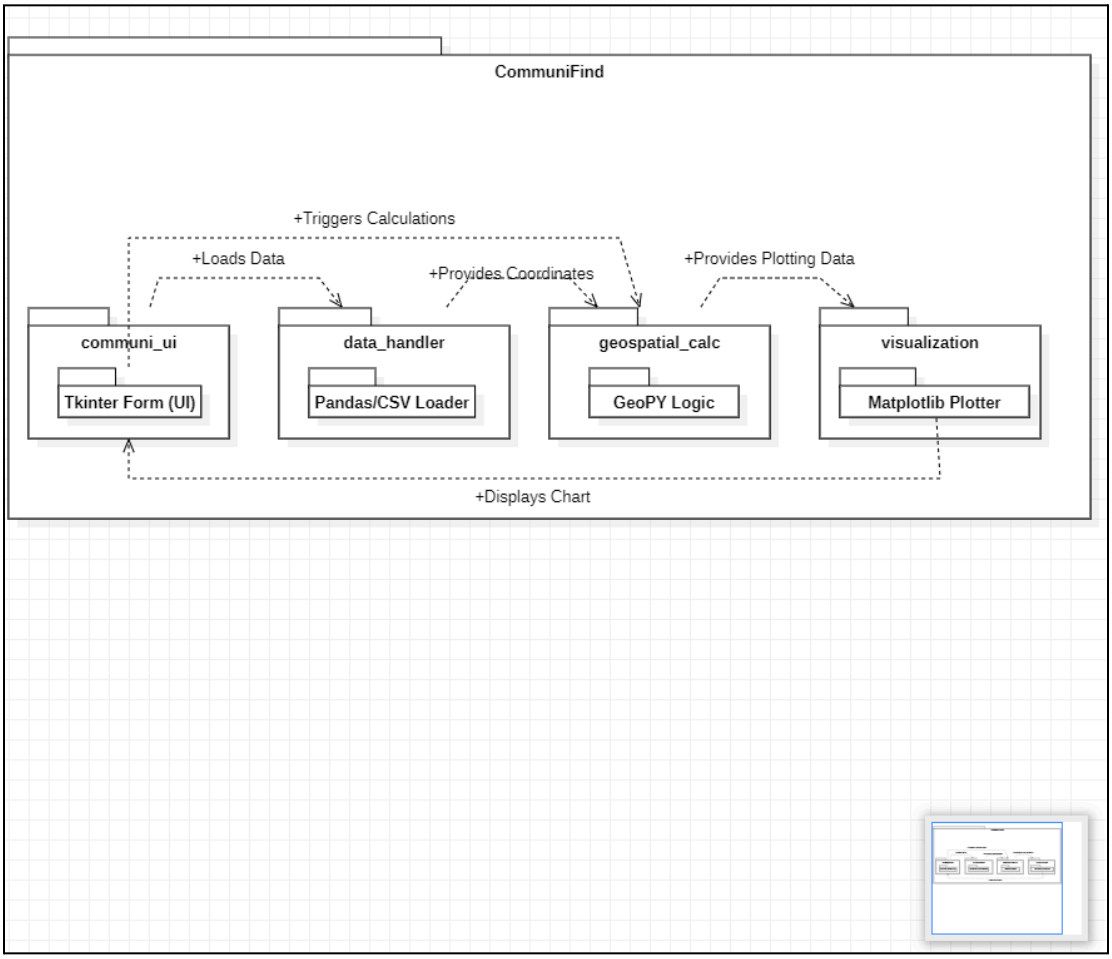
The diagram illustrates the three-tier architecture of the system "CommuniFind." The User Device (UI) communicates with the Application Server (Backend API) using REST Requests. The Application Server then accesses the Database Server MongoDB (NoSQL) using SQL Queries to manage data.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

6. Package Diagram:

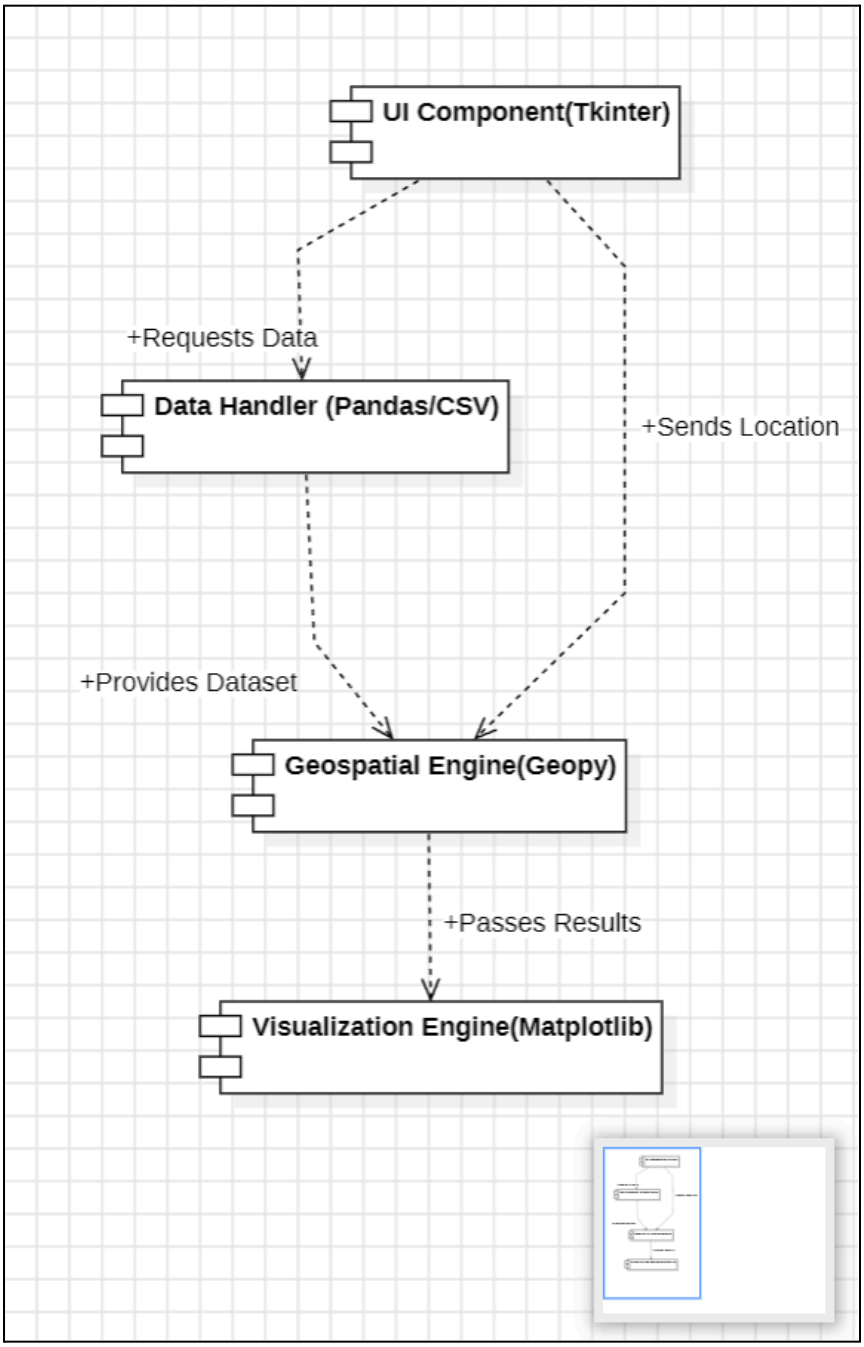
This package diagram represents the modular architecture of the *CommuniFind* system, showing how the application is organized into distinct functional packages. The communi_ui package interacts with the data_handler to load data, sends coordinates to geospatial_calc for processing, and receives processed results for visualization. The visualization package uses the computed geospatial data to generate charts, which are then displayed back to the user through the UI.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

7. Component Diagram:

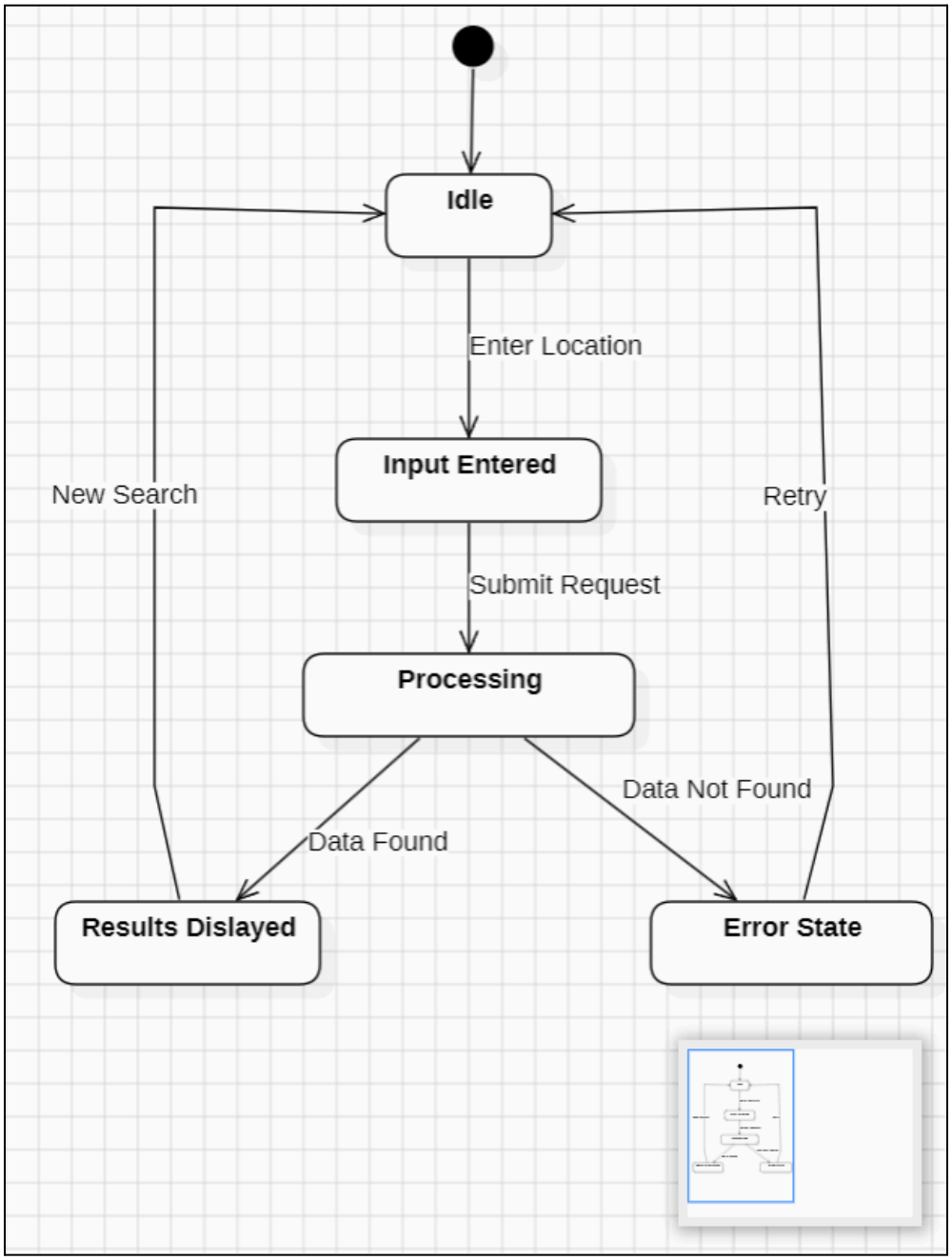
This component diagram illustrates the high-level software components of the *CommuniFind* system and their interactions. The UI Component requests data from the Data Handler, sends location information to the Geospatial Engine, and receives processed results. The Visualization Engine uses these results to generate visual outputs that are displayed back through the UI.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

8. State Machine Diagram:

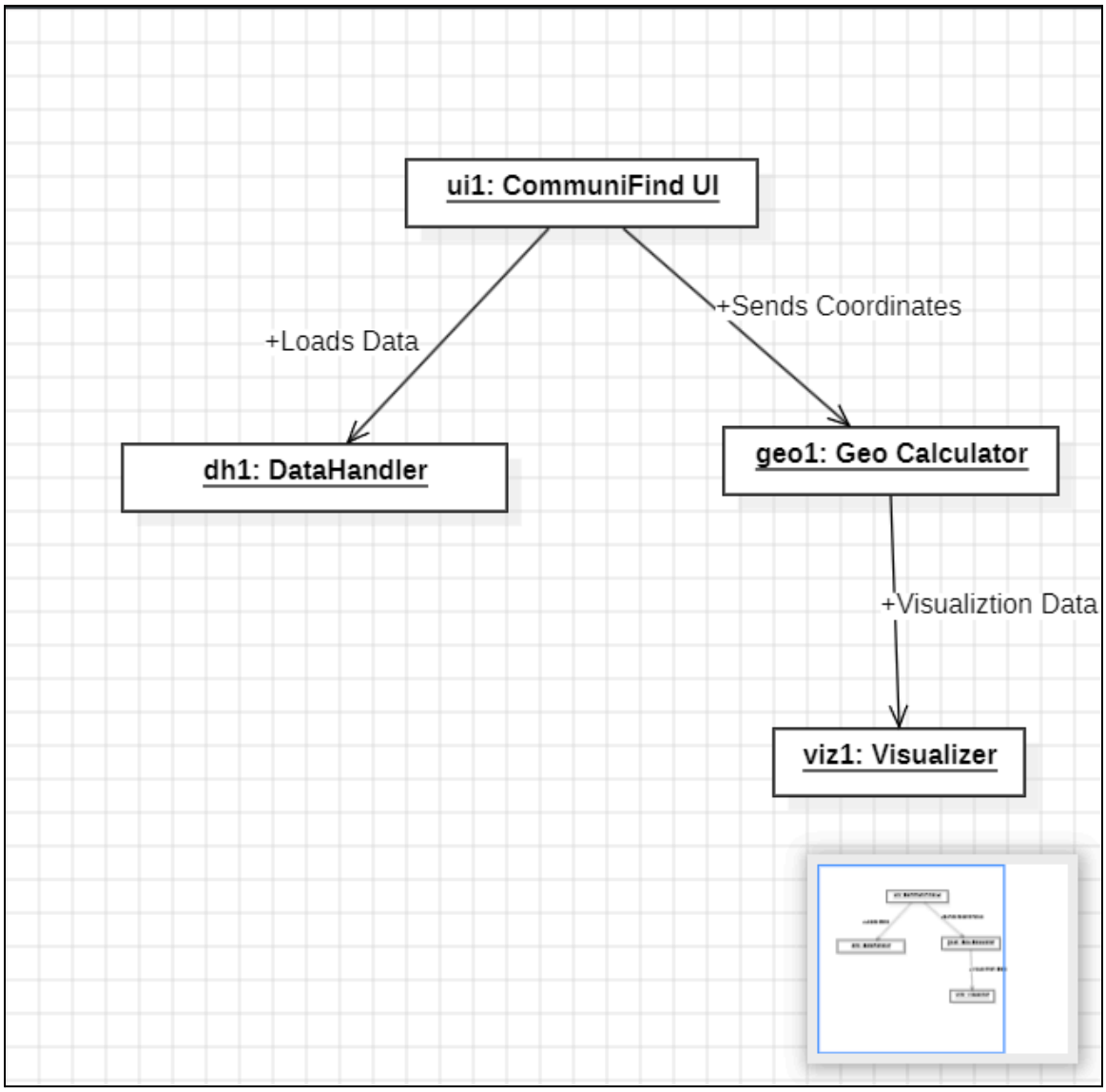
This state machine diagram models the behavior of the *CommuniFind* application during a search operation. The system starts in the Idle state, moves to Input Entered and Processing as the user submits a location request, and then transitions based on whether data is found or not. Successful processing leads to Results Displayed, while failures move the system to an Error State with options to retry or start a new search.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

9. Object Diagram:

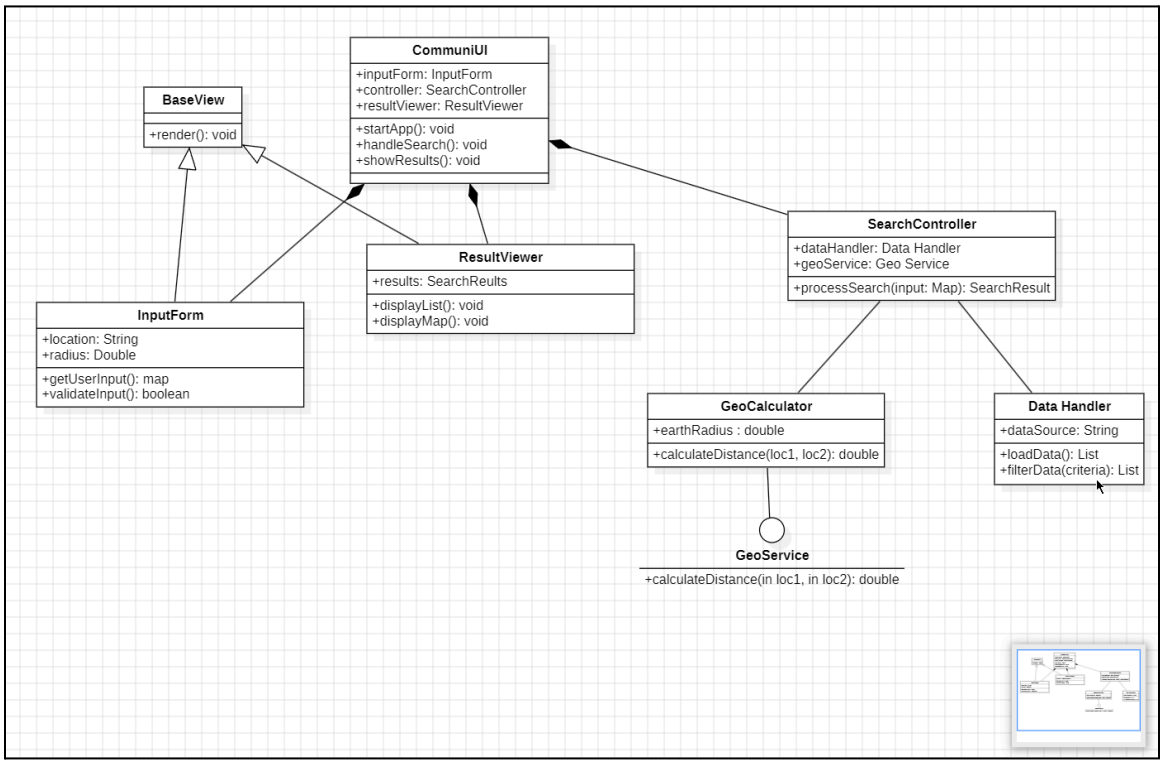
This object diagram represents a runtime snapshot of the *CommuniFind* system, showing instances of key objects and their relationships. The ui1 object interacts with dh1 to load data and sends coordinates to geo1 for geospatial processing. The processed visualization data is then passed to viz1, illustrating how objects collaborate during execution.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

10. Composite Structure Diagram:

This composite structure diagram shows the internal organization of the CommuniUI component and how its parts collaborate to perform a search operation. The UI is composed of InputForm, SearchController, and ResultViewer, which handle user input, request processing, and result presentation respectively. Supporting components such as DataHandler and GeoCalculator provide data access and geospatial services through defined interfaces.



Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

5. IMPLEMENTATION

Snippet 1: Connecting to MongoDB & Creating Index

```
# Database Setup (from database_setup.py)
def setup_db():
    client = pymongo.MongoClient("mongodb://localhost:27017/")
    db = client["CommuniFindDB"]

    # Create geospatial index for 2dsphere queries
    db.locations.create_index([("coordinates", "2dsphere")])
    return db.locations
```

Snippet 2: Fetching Data from Overpass API

```
# Fetching Logic (from database_setup.py)
query = f"""
[out:json][timeout:30];
(
  node["amenity"](around:2000,{coords[0]},{coords[1]});
  node["shop"](around:2000,{coords[0]},{coords[1]});
);
out body;
"""
```

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

Snippet 3: The Haversine Bearing Calculation

```
# Compass Bearing Calculation (from backend.py)
def get_bearing(lat1, lon1, lat2, lon2):
    d_lon = math.radians(lon2 - lon1)
    # ... (trigonometric conversions) ...
    y = math.cos(lat1) * math.sin(lat2) - (math.sin(lat1) * math.cos(lat2) *
math.cos(d_lon))
    initial_bearing = math.atan2(x, y)

    # Normalize to 0-360 degrees
    compass_bearing = (math.degrees(initial_bearing) + 360) % 360
    return compass_bearing
```

Snippet 4: The 10km Radius Filter

```
# Search Filtering Logic (from backend.py)
for index, row in matched_df.iterrows():
    loc_coords = (row['lat'], row['lng'])

    # Calculate geodesic distance
    dist_meters = geodesic(user_coords, loc_coords).meters

    # Filter: Exclude results > 10km
    if dist_meters > 10000:
        continue
```

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

Snippet 5: Converting Geo-Coordinates to Cartesian (X/Y)

```
# Relative Coordinate Conversion (from backend.py)
# Converts Lat/Lng to Meters relative to User (0,0)
rel_x = (loc["lng"] - user_lng) * 111000 * math.cos(math.radians(user_lat))
rel_y = (loc["lat"] - user_lat) * 111000
```

Snippet 6: Interactive Map Drawing

```
# Visualization Logic (from main.py)
def draw_map(highlight_idx=None):
    ax.clear()
    # Plot User at Center (0,0)
    ax.scatter([0], [0], color=COLOR_PRIMARY, s=150, label="You", marker="^")

    if highlight_idx is not None:
        # Draw Line to Target
        ax.plot([0, tx], [0, ty], color=COLOR_WARN, linestyle='-', linewidth=2)

        # Add Distance Label
        ax.text(mid_x, mid_y, f"{dist}m {bearing}", fontsize=9, color='blue')
```

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

6. RESULTS

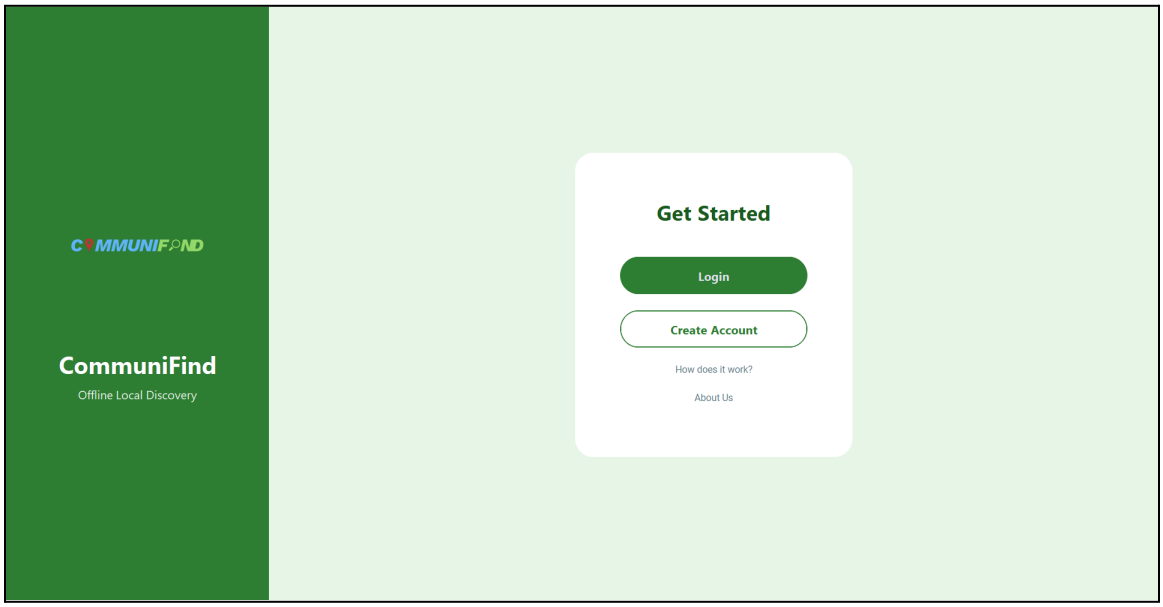


Figure 12: Application Landing Page providing user authentication options.

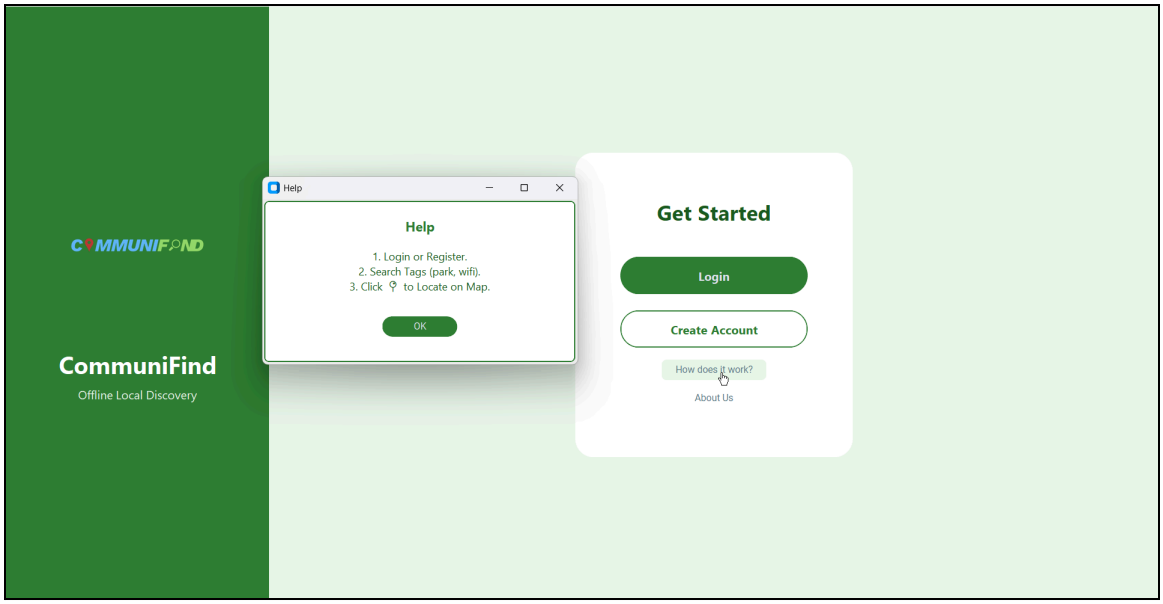


Figure 13: Built-in User Guide explaining application functionality.

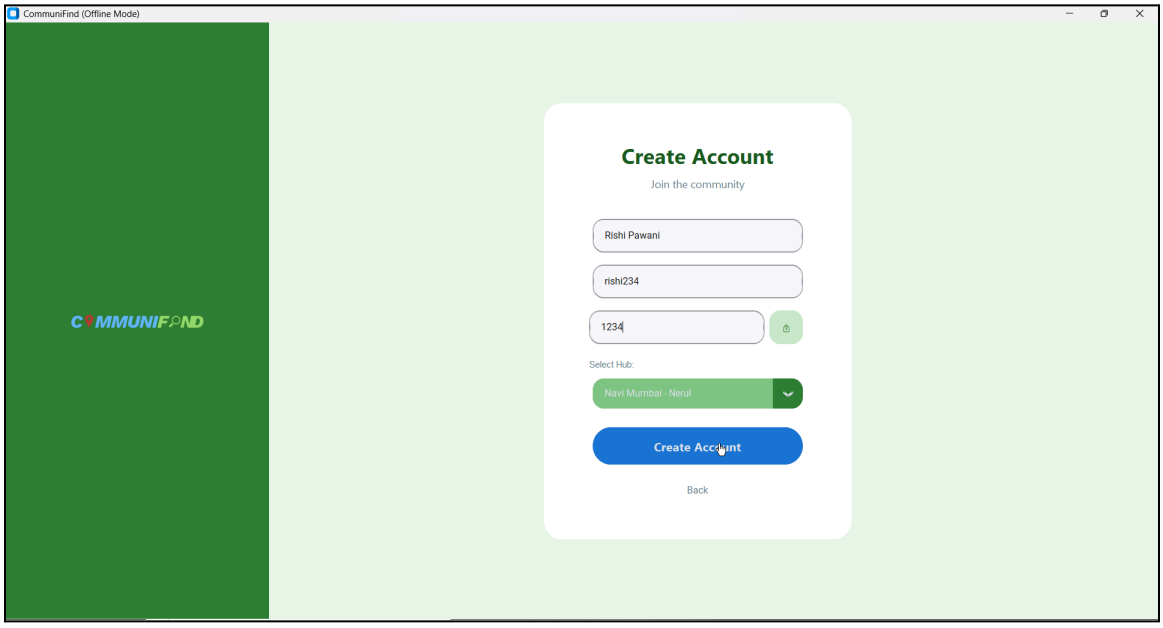


Figure 14: User Registration Module with Region Selection.

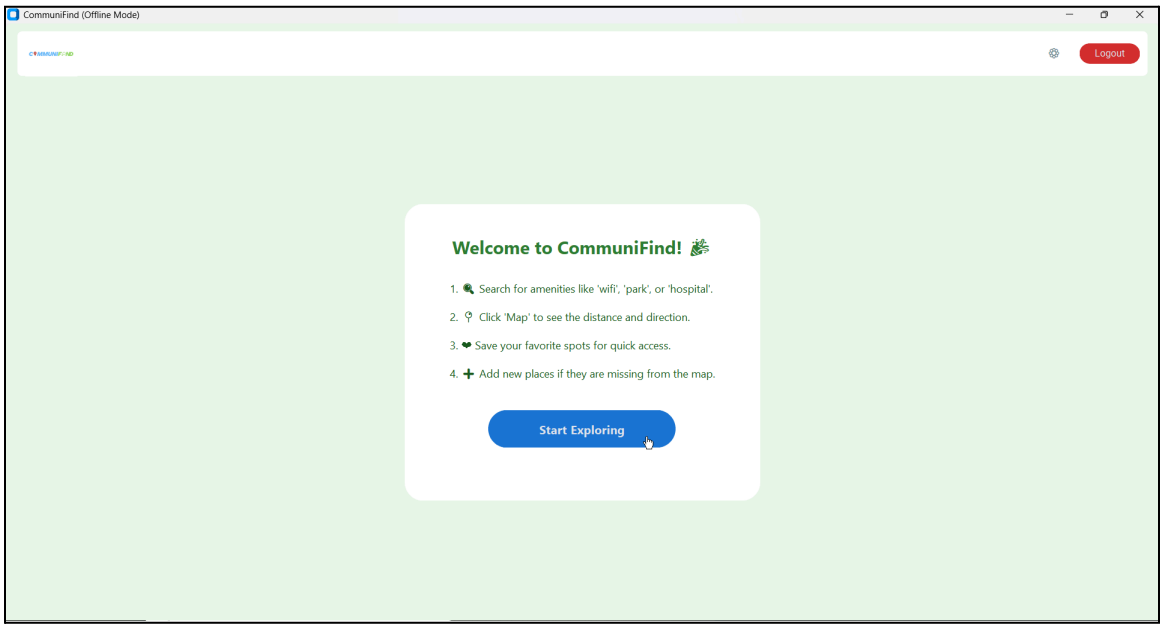


Figure 15: Onboarding Tutorial for first-time users

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

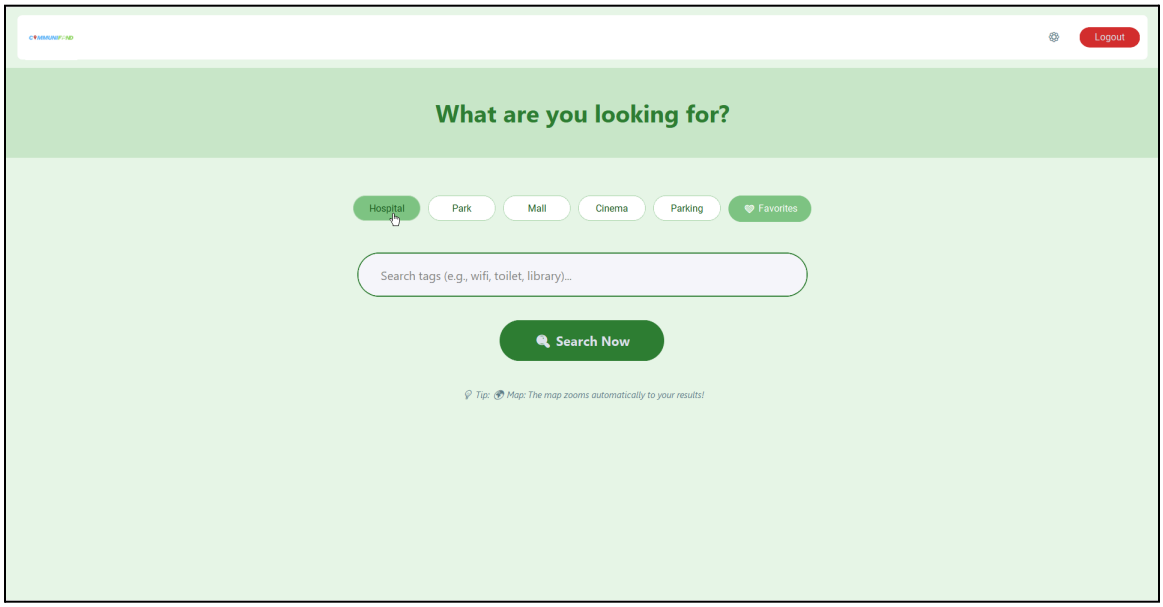


Figure 16: Home Dashboard showing Search Interface and Dynamic Safety Tips.

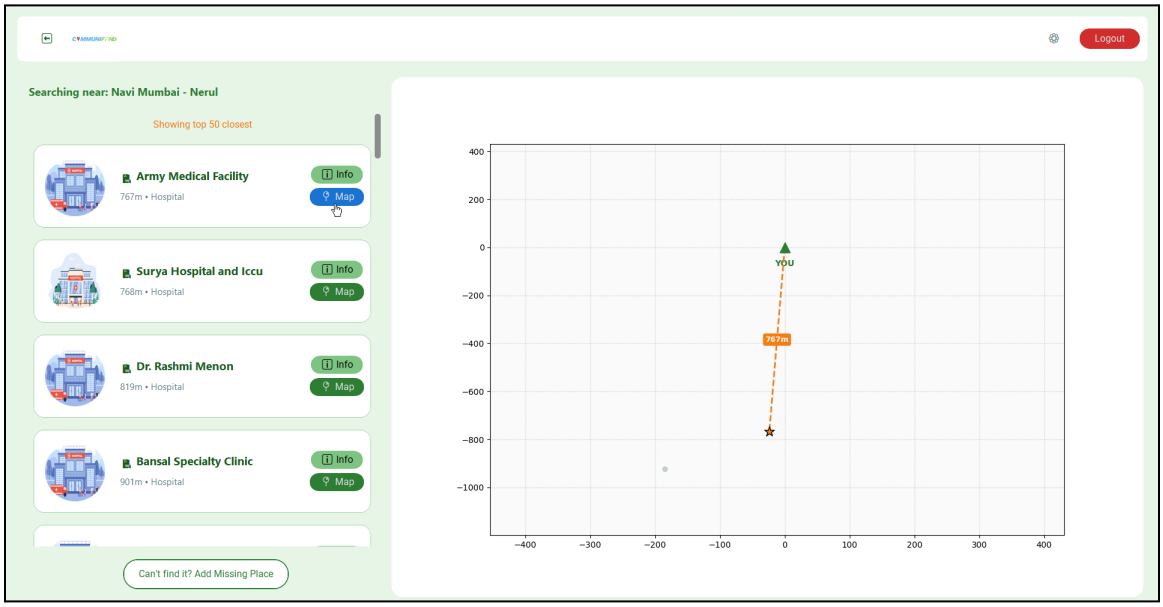


Figure 17: Search Results illustrating the Relative Plotting Algorithm with distance and bearing calculation.

Figure 18: Error Handling and User Feedback mechanism.

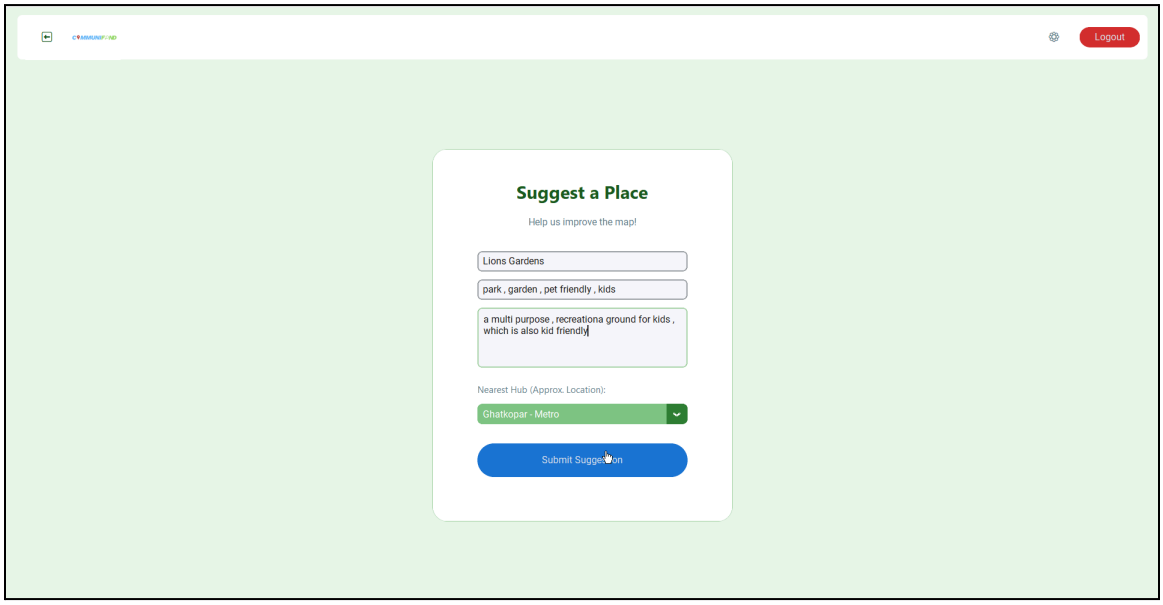


Figure 19: Crowdsourcing Interface allowing users to contribute data.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

Figure 20: Administrator Panel for moderating user-submitted location requests.

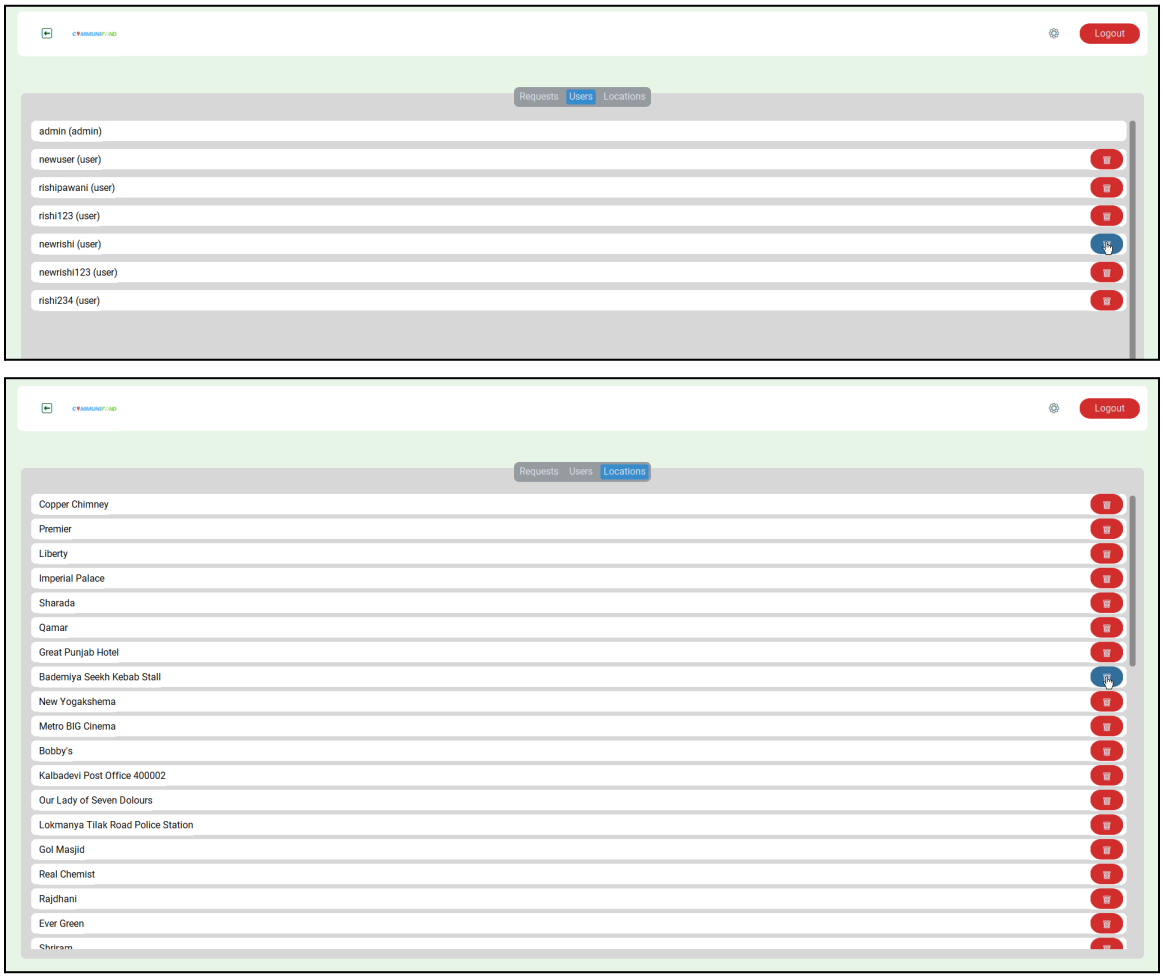


Figure 21: Administrative controls for managing registered users and location databases.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

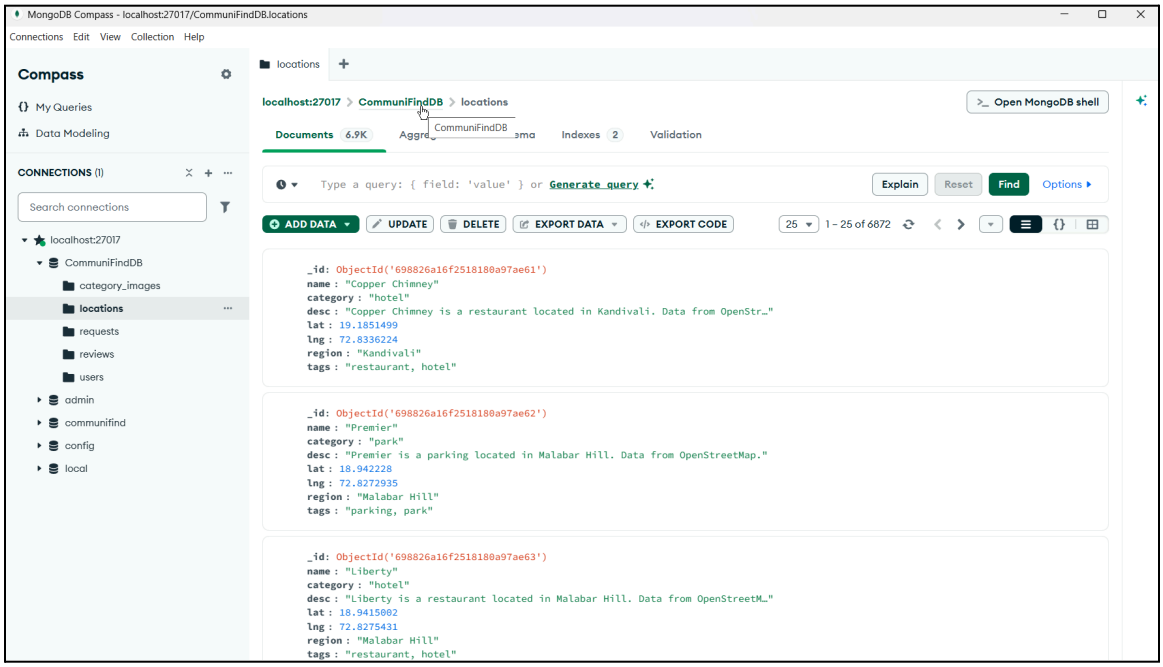


Figure 22: The CommuniFind backend database schema visualized in MongoDB Compass.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

7. CONCLUSION

The "CommuniFind" project successfully demonstrates the feasibility and utility of an offline, hyper-local feature locator tailored for the complex urban environment of Mumbai. By shifting the focus from commercial points of interest to community-centric amenities, the application bridges the "granularity gap" identified in mainstream mapping platforms like Google Maps.

The development process validated that a robust Python-based architecture, utilizing **CustomTkinter** for a modern interface, **MongoDB** for flexible data storage, and **Geopy** for geospatial calculations, is sufficient to provide accurate, real-time navigation data without requiring continuous internet connectivity. The system effectively ensures user privacy by maintaining all location data and user preferences on the local machine.

Furthermore, CommuniFind goes beyond simple static mapping by successfully implementing dynamic features such as **crowdsourcing**, which allows users to contribute new local data, and **user personalization**, enabling the saving of favorite locations. This proves that digital navigation tools for Indian megacities do not need to be complex, cloud-dependent giants. Instead, a targeted, data-rich, and community-driven approach can better serve the specific needs of local citizens, empowering them to locate and share essential resources like public toilets, wifi zones, and cultural landmarks with ease and precision.

FUTURE DEVELOPMENTS

While the current version of CommuniFind serves as a fully functional desktop application, several enhancements have been identified to further increase its utility and reach:

1. **Vernacular Language Support (Translation Layer):** Integration of a translation module to support local terminology. For example, allowing a user to search for "Sauchalaya" or "Arogya Kendra" in Marathi/Hindi, which the system effectively maps to the English tags "Toilet" or "Health Center" before filtering.
2. **Mobile Application Porting:** Migrating the codebase from a desktop-based Python framework to a mobile framework (such as Kivy, React Native, or Flutter) to allow the application to run on Android smartphones, making it truly portable for users actively navigating the streets.
3. **Live GPS Integration:** While the current version protects privacy by using manual location selection, a future "Online Mode" could optionally access the device's real-time GPS sensor for automatic location detection when required.
4. **Cloud Synchronization:** Developing a secure sync mechanism that allows the locally cached "Crowdsourced" data to be uploaded to a central server when internet access is available, allowing the community database to grow collectively across all users.

Name : Rishi Pawani
Class : SYIT A
Roll no. : 5551

REFERENCES / BIBLIOGRAPHY

1. Technical Documentation & Libraries

1. Python Software Foundation. *Python 3.12.1 Documentation: Math Module*. Full URL: https://docs.python.org/3/library/math.html
2. Schimansky, T. *CustomTkinter: A modern and customizable python UI-library based on Tkinter*. GitHub Repository: https://github.com/TomSchimansky/CustomTkinter
3. Hunter, J. D. "Matplotlib: A 2D Graphics Environment". *Computing in Science & Engineering*, Vol. 9, No. 3 (2007), pp. 90-95. Full URL: https://matplotlib.org/stable/users/index.html
4. Clark, A. (Pillow). *Pillow (PIL Fork) Documentation*. Full URL: https://pillow.readthedocs.io/en/stable/

2. Algorithms & Mathematics

5. Vennard, M. "The Haversine Formula: Calculating Distance Between Two Points on a Sphere". *MathForum Archives*. Full URL: http://mathforum.org/library/drmath/view/51879.html

6. Movable Type Scripts. *Calculate distance, bearing and more between Latitude/Longitude points*. Full URL: https://www.movable-type.co.uk/scripts/latlong.html *(Reference for the bearing calculation logic used in the application)*

3. Data Sources

7.OpenStreetMap. *Coordinates for Mumbai Locations (Dadar, Bandra, Colaba)*. Full URL: https://www.openstreetmap.org/ *(Source used for verifying the hardcoded coordinates in MUMBAI_HUBS)*

8. Brihanmumbai Municipal Corporation (BMC). *List of Public Amenities and Ward Offices*. Full URL: https://portal.mcgm.gov.in/